Step-Level LLM Routing for Software Engineering Agents

Tobias Lindenbauer, Kirill Gelvan, Alexander Kovrigin, Veronika Smilga, Igor Slinko, Konstantin Grotov, Yaroslav Zharov

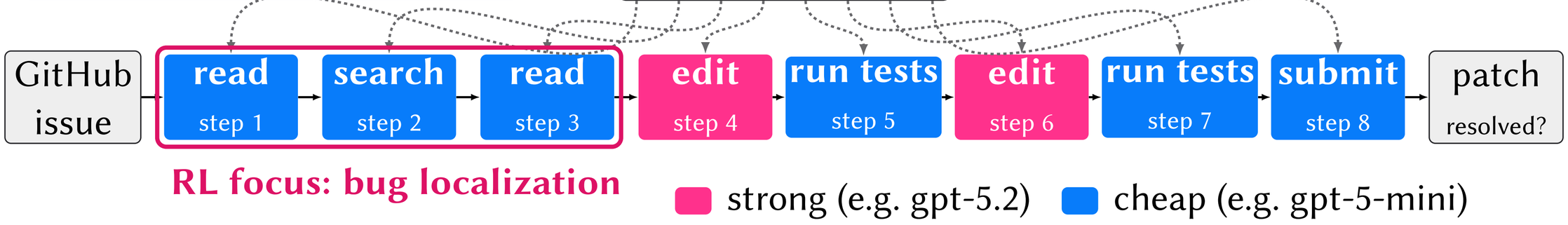
Different steps of one agent run can use different LLMs: **cheaper at the same quality**.
Headroom is large, but a router only realizes it with **privileged signal** (SFT) or **forced exploration** (RL).

Step-level routing: one harness (mini-swe-agent), a different LLM per step

SFT approach
Gold-patch + lookahead router labels the best model per step; **train on it**

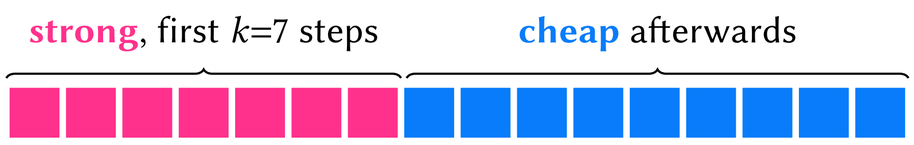
Router
picks the LLM for **each next step**

RL approach
Router LLM in a cost-aware env.; reward = outcome $- \lambda \cdot$cost



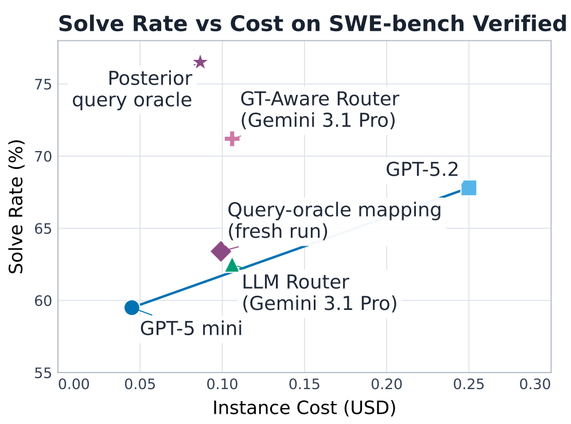
SFT approach: data & baselines

Cascading: The Baseline To Beat



- **Large headroom:** a per-turn oracle sits far above the per-query oracle on the cost/quality front.
- **Cascading** (strong → weak) is the best simple baseline; weak → strong is much worse.
- **Fragile:** only k=7 helps; $k \in \{3, 11, 15, 23\}$ fall below the interpolation line. Identical runs differ by ~4%.
- A **trained small router** or a **prompted frontier router** does not beat a small API model: the prefix carries little signal.

Privileged Information Routing Realizes Gains



- Router sees the **gold patch** and a **1-step lookahead**; budget $0.183 (median gpt-5.2 cost).
- **On par with gpt-5.2 at** ~59% **lower cost**; the cheap model takes ~50–60% of steps.
- GT-aware router uses the strong model selectively throughout the trajectory.

Privileged Information Distillation Does Not

Collect runs with the **GT-aware router** → Remove **gold patch and lookaheads** from context → Prepare dataset with rubrics only to **prevent leakage** → **SFT** a router

- Distilled 4B dense Qwen bases into router from on GT-aware data.
- **Failed to significantly or consistently outperform baselines.**
- Anticipating what a model may do next is a critical issues.
- **Other task formulations: LLMs as tools, sub-task decomposition**

RL approach: end-to-end training

Obstacle: routers collapse to one model

SWE-smith **bug localization**; routers: Qwen2.5-3B/7B, Qwen3-8B, and more.

1 model	~2×	≈ random
trained routers rarely switch	cheaper: random vs. strong-only	router: best level any trained router reaches

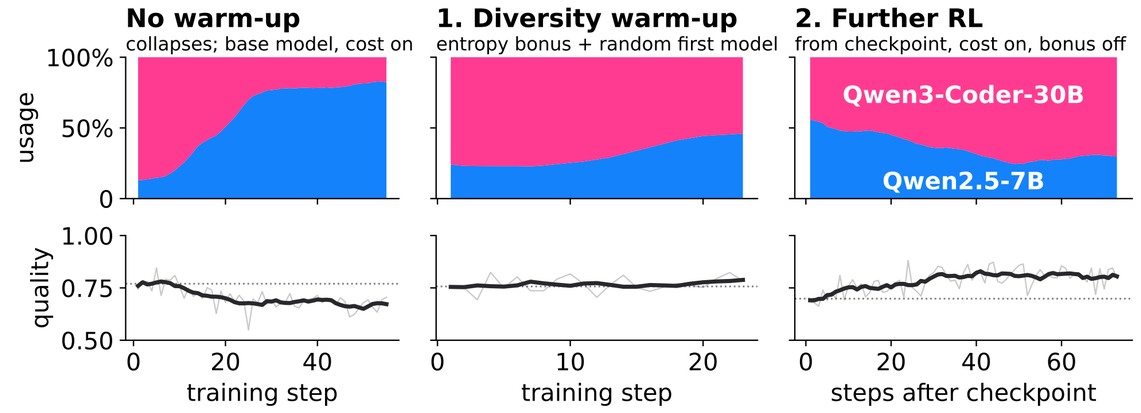
- Strong models **over-do** the task; routers **self-submit** and exit early.

What we tried to fix it

✓ **Entropy bonus**	On model mix + random first model: usage **becomes diverse**, outcome rises with entropy
≈ **λ sweep**	$\lambda \in \{0, 1, 10, 25, 100\}$: smooth shift strong → cheap; ≈ random-router Pareto front
≈ **Cost curricula**	Linear, step, delayed, range: stable learning; at best on par with random router
≈ **Adaptive budget**	Tighten to median cost of successes: beyond λ-front at a fixed budget (success ~80%); random level across budgets
✗ **Forced random prefix**	Alone: still sticks to the cheap model
✗ **SFT on random router**	Routing unbalanced even on balanced data; no generalization to unseen models, even with aliases

✓ works ≈ partial / on par with random ✗ does not work

Diversity first, then cost: training dynamics



Lessons

- **Exploration, not cost shaping, is the bottleneck:** fix model diversity first, then add cost.
- Very **sensitive** to λ, thresholds and schedules; evaluation is fragile.
- Routing signal is **implicit** here: start with more **verifiable** tasks.

JETBRAINS Research